自动舵控制算法 C++ 代码说明文档

> **文档编号**：AR-CODE-2026-001 **版本**：V1.0 **编制日期**：2026-07-29 **对应设计文档**：《自动舵定位与控制设计文档》 §3~§5 **代码目录**：`control/include/`

---

1. 概述

本文档说明自动舵 **控制部分** 的纯算法 C++ 实现，对应设计文档中的：

- §3.2~§3.7 PD 航向控制律
- §3.6 增益调度
- §3.9 控制算法标定（Z 形试验辨识 + PD 增益初算）
- §3.10 控制参数自动优化（RLS 在线辨识 + 极点配置 + ESC）
- §4 仅航向条件下的机动功能（Leg Sequence 框架）
- §5 定位与控制联合工作流（主循环集成）

纯算法层：不包含任何硬件 IO（GPIO/CAN/UART/EEPROM），所有传感器输入与舵机输出通过函数参数传递，便于单元测试与 HIL 仿真。

---

2. 代码结构

```
control/include/
├── angle_utils.hpp          # 角度归一化与差值
├── pd_controller.hpp        # PD 航向控制器
├── gain_schedule.hpp        # 增益调度表
├── nomoto_identifier.hpp    # Nomoto 模型 RLS 在线辨识
├── auto_tuner.hpp           # 极点配置 + ESC + 稳定性监控
├── maneuver_sequencer.hpp   # 机动航段序列框架
├── param_store.hpp          # 参数存储 + CRC32
└── autopilot_core.hpp       # 核心集成（主循环）
```

所有文件均为 **header-only**，C++17 标准，无第三方依赖，便于嵌入 ECU。

---

3. 模块详解

3.1 angle_utils.hpp — 角度归一化

对应文档：§3.2 角度归一化

函数	说明
`normalizeAngleRad(a)`	将角度归一化到 $(-\pi, \pi]$，弧度
`normalizeAngleDeg(a)`	将角度归一化到 $(-180, 180]$，度
`angleDiffRad(ref, act)`	最小带符号差 $ref - act$，弧度
`angleDiffDeg(ref, act)`	同上，度

用途：航向误差计算必须做角度归一化，否则 359° 与 1° 的误差会被误算为 358° 而非 -2°。

```
const double e = angleDiffDeg(headingRef, headingAct);  // 落在 (-180, 180]
```

3.2 pd_controller.hpp — PD 航向控制器

对应文档：§3.2 控制律、§3.3 限幅、§3.7 抗噪与死区

控制律

$$\delta_k = K_p, e_k + K_d, \dot{e}_k$$

关键特性

特性	实现位置	说明
角度归一化误差	`angleDiffDeg`	§3.2
微分低通滤波	`alpha = 1/(1+τ/dt)`	§3.7 一阶低通，τ=0.2 s
误差死区	` ` `	e
舵角限幅	`±dMax`	§3.3，默认 ±35°
舵速限幅	`ddMax · dt`	§3.3，默认 10°/s

特性	实现位置	说明
输出量化	`round(v/quant)·quant`	§3.7，默认 0.1°

使用

```
ar::PdController::Params p;
p.Kp = 1.5; p.Kd = 0.6;
p.dt = 0.02;  // 50 Hz
ar::PdController pd(p);

double rudderCmd = pd.update(headingRef, headingAct);
```

参数说明

参数	默认值	单位	说明
Kp	1.0	—	比例增益
Kd	0.5	—	微分增益
dMax	35.0	deg	舵角限幅
ddMax	10.0	deg/s	舵速限幅
deadzone	0.3	deg	误差死区
tau	0.2	s	微分滤波时间常数
quant	0.1	deg	输出量化步长
dt	0.02	s	控制周期

3.3 gain_schedule.hpp — 增益调度

对应文档：§3.6 / §3.10.4

按船速与海况查询 PD 增益，内置默认调度表（与文档 §3.10.4 一致）：

船速区间	海况	Kp	Kd
< 4 kn	0~1	2.0	0.8

船速区间	海况	Kp	Kd
4~10 kn	0~1	1.5	0.6
4~10 kn	2~3	1.0	0.4
10~18 kn	0~1	1.2	0.5
10~18 kn	2~3	0.9	0.4
18~25 kn	0~1	1.0	0.4
18~25 kn	2~3	0.7	0.3

```
ar::GainSchedule sched;
double Kp, Kd;
sched.query(speedKn, seaState, Kp, Kd);
```

3.4 nomoto_identifier.hpp — Nomoto 模型在线辨识

对应文档：§3.9.4 Z 形试验辨识 / §3.10.3 RLS 在线辨识

模型

$$T\dot{r} + r = K\delta \;\Rightarrow\; r_k = K\delta_k - \frac{T}{\Delta t}(r_k - r_{k-1})$$

RLS 递推

$$K_k = \frac{P_{k-1}\phi_k}{\lambda + \phi_k^T P_{k-1}\phi_k}$$

$$\hat{\theta}_k = \hat{\theta}_{k-1} + K_k(y_k - \phi_k^T \hat{\theta}_{k-1})$$

$$P_k = \frac{1}{\lambda}(P_{k-1} - K_k \phi_k^T P_{k-1})$$

其中 $\theta = [K, T]^T$，$\phi_k = [\delta_k, -(r_k - r_{k-1})/\Delta t]^T$，$y_k = r_k$。

激励条件

仅当 $|\delta| > \delta_{min}$ 且（$|r| > r_{min}$ 或 $|\dot{r}| > \dot{r}_{min}$）时更新（默认 $\delta_{min} = 3^\circ$、$r_{min} = 0.5^\circ/s$、$\dot{r}_{min} = 0.2^\circ/s^2$），避免无激励时的协方差爆炸；保留 r 过零附近的高 $\dot{r}$ 瞬态，否则 T 欠定。

自降频（R1 深化）

`update(δ, r, dtNow)` 可以任意频率调用；内部累计满 `Params.beatSec`（默认 1.0 s）才做一次 RLS 拍（点采样：取拍时刻的样本）。差分 Δt 为两次 RLS 拍的真实间隔，由模块自测，与调用频率无关。旧设计"调用方按 1 s 节拍调用 + 改写 `Params.dt`"的语义陷阱已随该字段一并删除。

使用

```
ar::NomotoIdentifier id;
id.update(rudderDeg, rateDegS, dt);  // 任意频率喂入；内部自降频，满 1 s 做一次 RLS 拍
double K = id.K(), T = id.T();
```

3.5 auto_tuner.hpp — 控制参数自动优化

对应文档：§3.10

三层架构

层	方法	函数
层 1 在线辨识	RLS	`NomotoIdentifier`（独立模块，以 `IdSnapshot` 快照传入）
层 2 极点配置	闭环 ζ/ωn 反推	`AutoTuner::polePlacement`（公有静态，公式全仓库唯一出处）
层 3 性能寻优	ESC 极值搜索	`AutoTuner::tick()` 内部驱动（私有 `escStep`）

接口结构体（R1 深化新增）

```
struct IdSnapshot { bool excited; bool valid; double K; double T; };  // 辨识快照
struct TunerInput { double headingRefDeg, headingDeg, rudderDeg, speedKn, dt,
                    schedKp, schedKd; IdSnapshot id; };              // tick() 每拍输入
struct GainCommand { double Kp, Kd; };   // tick() 每拍输出（已含 ESC 扰动载波）
```

公有动作（R1 深化后仅 3 个）

动作	说明
`tick(const TunerInput&) → GainCommand`	每拍单入口：速度门控 → 无基线时跟随调度值 → 周期辨识评估（P12 漂移检查）→ 参考稳定门控 + 性能样本累积（含因果解调）→ ESC 窗口结算（稳定门 + 回退）→ 扰动注入 → 返回增益
`establishBaseline(const IdSnapshot&) → bool`	一次性评估建基线（P14 辨识试验收官 / 人工标定注入）：不做漂移检查、振荡条件以 0 把关、恒重锚；与 `tick` 共享私有 `applyEstimate_()`
`suspend()`	唯一模式生命周期钩子：停用 + 清测量状态（窗口/解调/HPF/梯度），保留基线

只读查询

`enabled()`、`baselineEstablished()`、`currentGains(Kp&, Kd&)`（去载波投影，观测增益所有权）、`computeJ()`、`oscDetector()`。

私有机制（R1 起经 tick/establishBaseline 驱动）

- 既有机制私有化：`accumulate`（性能样本 + 因果解调）、`escDither`（扰动注入）、`escStep`（窗口结算梯度更新）、`revert`（回退基线）、`rebaseEsc`（重锚清零）、`setBaseline`
- R1 新增私有路径：`applyEstimate_`（周期评估与一次性评估的共享实现，driftGate / forceRebase 两路径）、`settleWindow_`（原 `AutopilotCore::escTrigger` 内收）
- 从 core 迁入的状态：`curKp_/curKd_`（增益所有权，基线建立后 ESC 拥有，不再跟随调度表）、`baseK_/baseT_`（漂移判据基准）、`winCounter_`、`refAgeSec_/prevRef_`（振荡检测门控）、`baselineEstablished_`；P12 策略常量随迁为私有 constexpr（`kReBaselineTol=0.25` 等）

极点配置（层 2）

$$K_p = \frac{\omega_n^2 T}{K}, \quad K_d = \frac{2\zeta\omega_n T - 1}{K}$$

```
double Kp, Kd;
AutoTuner::polePlacement(K, T, 0.85, 0.20, Kp, Kd);
```

性能指标（ITAE）

$$J = \int_0^{T_w} \left(t|e| + w_u|\delta| + w_{du}|\dot{\delta}| \right) dt$$

ESC 更新律

$$K_{p,k+1} = K_{p,k} - \eta, \widehat{\partial J / \partial K_p}$$

通过对 K_p, K_d 加正弦扰动（相位错开 90° 避免耦合），解调估计梯度。

稳定性监控（§3.10.7）

```
bool ok = tuner.stabilityCheck(K, T, Kp, Kd, oscAmplitude);
// 条件：K/T 有限非退化、Kp > 0、ζ ⩾ 0.6、ωn ⩽ 0.5、振荡 ⩽ 5°
// （单次更新幅值 ⩽ 10% 是 ESC 积分更新处私有 clampUpdate 的约束，不在本检查内）
```

安全约束

约束	阈值
单次更新幅值	⩽ 10%
高速去激活	> 15 kn
稳定性四条件	全满足才接受
回退	窗口结算未过稳定门时恢复基线（私有 `revert()`，由 `tick()` 触发）

3.6 maneuver_sequencer.hpp — 机动航段序列

对应文档：§4.2 通用机动调度框架

Leg 结构

```
struct Leg {
    Mode    mode;       // HEADING / RUDDER
    double  target;     // 目标航向 或 目标舵角
    Trigger trigger;    // HEADING_REACHED / TIME / RATE / NONE
    double  threshold;  // 触发阈值
    int     next;       // 下一段索引，-1 结束
};
```

预置机动

函数	对应文档	说明
`williamson(psi0)`	§4.4	+δ_max 至 Δψ=60° → -δ_max 至 Δψ=180° → 保持
`uTurn(psi0)`	§4.5	PD 跟踪 ψ0±180°
`zigzag(psi0)`	§4.6	±δ_z 与 ±ϕ_z 交替 N 次
`circles()`	§4.3	恒舵角至累计 360°
`cloverleaf(psi0)`	§4.7	4×(转向 270° + 直航 t_leg)
`search(psi0)`	§4.8	扩张方搜，边长递增

使用

```
auto legs = ar::ManeuverSequencer::williamson(psi0);
ar::ManeuverSequencer seq;
seq.setLegs(legs);
while (!seq.done()) {
    auto cmd = seq.update(headingAct, rateAct, dt);
    // cmd.isHeading ? PD 跟踪 cmd.heading : 直接输出 cmd.rudder
}
```

3.7 param_store.hpp — 参数存储

对应文档：§3.9.8 / §3.10.9

ControlParams 结构

字段	对应 EEPROM 区块
rudderZero/Left/Right, ddotMax	区块 A
K, T	区块 B
Kp, Kd	区块 C
version, timestamp, crc	元信息

CRC32

采用 IEEE 802.3 多项式 `0xEDB88320`，校验除 crc 字段外的全部内容。

IO 注入

```
ar::ParamStore store(readFn, writeFn);  // 由 BSP 注入
ar::ControlParams p;
if (!store.load(p)) { /* CRC 失败，加载默认 */ }
p.Kp = 1.5;
store.save(p);  // 自动 version+1 + CRC
```

3.8 autopilot_core.hpp — 核心集成

对应文档：§5 定位与控制联合工作流

模式

模式	说明
MANUAL	透传舵角反馈
AUTO_HEADING	航向保持（PD + 增益调度 + 自动优化）
AUTO_MANEUVER	机动序列执行
HOLD	保持当前舵角

主循环

```
ar::AutopilotCore ap;
ap.setMode(ar::Mode::AUTO_HEADING);
ap.setHeadingRef(128.5);

while (running) {
    ar::SensorInput s = readSensors();
    double rudderCmd = ap.step(s, 0.02);
    writeRudder(rudderCmd);
}
```

R1 深化：编排权移交 AutoTuner

自动优化的编排（节拍计数、ESC 窗口、速度门控、增益所有权、稳定门、回退、重锚）全部收进 `AutoTuner::tick()`；本类回归本职——模式状态机 + 机动编排 + 每拍转发。删除 `runEscTuning()`、`tunerMut()`、`escTrigger()` 与 9 个编排成员（辨识节拍计数器、ESC 窗口计数器、ESC 增益副本、基线 K/T、参考变化计时、基线标志），构造函数里的辨识器 `dt` 改写一并移除（辨识器已自降频）；新增 `setTunerParams()` 整组注入调参参数（替代暴露 mutable 引用的旧后门）。`setMode` 切出到 MANUAL/HOLD 时调 `tuner_.suspend()`（清测量状态、保留基线）。文件由 346 行收敛至 202 行。

主循环内部流程

```
step(s, dt):
  ├── MANUAL: 透传
  ├── AUTO_HEADING: headingControlStep（约 15 行）
  │    ├── 首拍以实际舵角初始化舵速限幅基准
  │    ├── 增益调度查询 Kp/Kd
  │    ├── identifier_.update(δ, r, dt)——自降频，每拍无条件喂
  │    ├── tuner_.tick({...}) → GainCommand（R1：编排全部内收）
  │    ├── pd_.setGains(g.Kp, g.Kd)
  │    └── pd_.update → 目标舵角
  ├── AUTO_MANEUVER:
  │    ├── 序列推进；辨识试验期间喂辨识器
  │    ├── 试验收官 finalizeIdentTrial()——一行 establishBaseline 映射 OK/FAILED
  │    ├── HEADING 段：增益调度查询 Kp/Kd → setGains → PD 跟踪（P13）
  │    └── RUDDER 段直接输出
  └── HOLD: 冻结上一拍舵角指令
```

---

4. 数据流与接口

4.1 输入

```
struct SensorInput {
    double headingDeg;   // AHRS 艏向
    double rateDegS;      // AHRS 转向率
    double rudderDeg;     // 舵角反馈
    double speedKn;       // 船速
    int    seaState;      // 海况 0..3
};
```

4.2 输出

目标舵角（deg），由外部驱动层转换为 PWM/CAN 指令。

4.3 调用周期

模块	周期	说明
`pd_controller`	20 ms (50 Hz)	主控制环
`nomoto_identifier`	每拍喂入（内部自降频至 1 s）	R1：任意频率调用，累计满 `beatSec` 才做一次 RLS 拍
`auto_tuner.tick`	20 ms	R1：每拍单入口（速度门控/辨识评估/样本累积/扰动注入）
`auto_tuner` ESC 窗口结算	60 s (窗口)	R1：`tick()` 内部按 `windowSec/dt` 计数，私有 `settleWindow_` 结算
`maneuver_sequencer`	20 ms	段推进

---

5. 与设计文档的映射

设计文档章节	代码模块	关键函数
§3.2 控制律	pd_controller	`update()`
§3.3 限幅	pd_controller	内部 clip

设计文档章节	代码模块	关键函数
§3.6 增益调度	gain_schedule	`query()`
§3.7 抗噪死区	pd_controller	滤波/死区/量化
§3.8 稳定性分析	auto_tuner	`stabilityCheck()`
§3.9.4 Z 形辨识	nomoto_identifier	`update()`
§3.9.5 PD 增益初算	auto_tuner	`polePlacement()`
§3.9.8 参数存储	param_store	`save()/load()`
§3.10.3 RLS 辨识	nomoto_identifier	`update()`
§3.10.4 极点配置	auto_tuner	`polePlacement()`
§3.10.5 ESC 寻优	auto_tuner	`tick()`（私有 `escStep` 结算）
§3.10.6 触发逻辑	auto_tuner	`tick()`（速度门控内收，R1）
§3.10.7 稳定性监控	auto_tuner	`stabilityCheck()`（回退经 `tick()` 内私有 `revert`）
§4.2 机动框架	maneuver_sequencer	`setLegs()/update()`
§4.3~§4.8 预置机动	maneuver_sequencer	`williamson()/uTurn()/...`
§5 联合工作流	autopilot_core	`step()`

---

6. 编译与测试

6.1 编译要求

- C++17
- 无第三方依赖
- header-only，直接 `#include` 即可

6.2 单元测试

现有 90 个用例（8 个测试文件），Release + ASAN 双工具链全绿；`ar_sim --validate` 仿真验证矩阵与 R1 前逐项一致。R1 后调参器与 core 的交互经 `tick()` 行为测试断言（测试与调用方跨同一条 seam，无测试后门），如 `SelfDecimatesAtAnyCallRate`、`NoBaselinePassesScheduleGainsThrough`、`WindowSettleUpdateIsClamped`、`SettleRevertsToBaselineWhenEstimateInvalid`、`SettleAcceptsUpdateWhenEstimateHealthy`、`RebaselineClearsIntegratorState`。

模块	测试要点	用例数
angle_utils	跨 0°/360° 边界误差	7
pd_controller	阶跃响应、限幅、死区	12
gain_schedule	跨段查询、海况限幅	7
nomoto_identifier	已知 K/T 数据收敛性、任意调用率自降频	9
auto_tuner	极点配置精度、ESC 收敛、tick 行为（基线/窗口结算/重锚）	17
maneuver_sequencer	各预置机动段推进	14
param_store	CRC 校验、版本递增	7
autopilot_core	模式状态机、机动编排、每拍转发	17

6.3 HIL 集成

`AutopilotCore::step()` 是唯一入口，注入 `SensorInput`、输出目标舵角，与船舶六自由度模型闭环即可进行 HIL 仿真。

---

7. 安全约束实现汇总

约束	实现位置	说明
舵角限幅 ±35°	`PdController::update`	§3.3
舵速限幅 10°/s	`PdController::update`	§3.3
误差死区 0.3°	`PdController::update`	§3.7
微分低通 τ=0.2 s	`PdController::update`	§3.7

约束	实现位置	说明
RLS 激励门限	`NomotoIdentifier::update`	§3.10.3
单次更新 ≤ 10%	`AutoTuner::tick`（私有 `clampUpdate`）	§3.10.7
高速去激活 > 15 kn	`AutoTuner::tick`（速度门控内收，R1）	§3.10.6
稳定性四条件	`AutoTuner::stabilityCheck`	§3.10.7
回退基线	`AutoTuner::tick`（私有 `revert`，窗口结算触发）	§3.10.7
参数 CRC32	`ParamStore::save/load`	§3.9.8
版本号递增	`ParamStore::save`	§3.10.9
模式切换	`AutopilotCore::setMode`（切出自动模式时 `AutoTuner::suspend`，R1）	§5.2

---

8. 扩展接口

扩展点	接口	说明
自定义机动	`ManeuverSequencer::setLegs`	注入 Leg 序列
自定义增益调度	`GainSchedule` 构造	替换默认表
EEPROM IO	`ParamStore(ReadFn, WriteFn)`	注入 BSP 读写
性能指标权重	`AutoTuner::Params`	调整 ITAE 权重
极点配置目标	`AutoTuner::Params.zeta/wn`	按船速分段
控制周期	`PdController::Params.dt`	适配不同 ECU

— 文档结束 —